BURP WORKBENCH KR

0.25.0 처음 시작하는 웹 보안 점검

요청 하나를 고르고, 필요한 검사만 실행하고, 확인한 근거를 저장하는 방법

처음에는 모든 기능을 사용할 필요가 없습니다. 2쪽에서 목적을 고른 뒤 해당 쪽으로 이동하세요. 처음 Burp를 쓰면 3~5쪽의 용어·설치·짧은 실습부터 시작합니다. 로그인 없는 첫 검사는 5쪽 뒤 바로 8쪽으로 가세요. 계정 준비는 계정 비교를 할 때 필요합니다.

이 안내서의 기준	내용
확장 / 실습	Workbench KR 0.25.0 / LOOP Community Lab 3.5.0
가장 짧은 시작	사이트 접속 → 요청 선택 → 범위 등록 → 응답 확인
계정 비교 예제	Bob 소유 비공개 글 102를 Alice와 비교
화면	현재 운영 Swing 코드와 가상 데이터로 만든 미리보기
실행 확인	해당 버전 검증 기록과 PR의 실제 검사 결과

화면 읽기: 파란 제목은 기능, 번호는 클릭 순서, '보여야 할 결과'는 다음 단계로 넘어가기 전 확인할 사항입니다. 예시 화면은 실제 Burp 창 캡처가 아니며, 화면에 보이는 데이터는 검증 완료 기록이 아닙니다.

확장 하단의 기능 안내에서 처음 시작하기 또는 기능 이름을 고릅니다. 용도·순서·추가 전송 여부를 읽고 이 기능 열기로 이동할 수 있습니다. 패널 사이 분할선을 끌어 요청·응답을 넓혀 보세요.

01 · 지금 무엇을 하고 싶나요?

목적에 맞는 기능 하나부터 시작합니다. '후보 찾기'와 '실제로 검사하기'는 다른 작업입니다.

하고 싶은 일	열 화면	이 안내서
사이트가 주고받는 내용 보기	요청·응답 목록	5쪽
다른 계정도 글을 읽을 수 있는지 비교	계정 권한 비교	6~7쪽
검색어 같은 입력값을 바꿔 검사	입력값 보안 검사	8쪽
JavaScript에서 API 경로 찾기	JS에서 API 찾기	9쪽
서버 단서와 알려진 경로 확인	서버·경로 탐색	10쪽
과거·관측 URL을 점검 후보로 모으기	URL 후보 수집	11쪽
아직 점검하지 않은 URL·계정 찾기	점검 진행 현황	12~13쪽
응답 헤더·오류·API 문서 확인	응답 보안 분석	14~15쪽
저장한 값이 나중에 어디에 나오는지 확인	응답 보안 분석 → 저장/조회 연결	16쪽
인코딩·JWT·두 응답 차이 보기	데이터 변환·비교	17쪽
여러 요청 순서대로 재현·증거 저장	재현·증거 정리	18~21쪽
업무 완료와 다음 재검토 정리	점검 진행 현황 → 업무별 현황	22쪽
익숙한 이전 이름으로 기능 찾기	이전 이름 → 새 이름 대응표	24쪽

팁: 탭을 여는 것과 검사 실행은 다릅니다. 자동 관찰은 이미 받은 응답을 읽습니다. '시작', '비교 실행', '순차 실행' 같은 실행 버튼은 대상에 요청을 보낼 수 있습니다.

02 · 요청·응답·계정을 먼저 이해하기

요청은 브라우저가 서버에 보내는 질문, 응답은 서버가 돌려주는 답입니다. 예를 들어 GET /posts/102는 '102번 글을 보여 달라'는 요청입니다.

화면 용어	쉽게 읽으면
메서드 / URL	하려는 작업 / 작업할 주소. GET은 주로 조회, POST는 주로 입력
쿠키·토큰 / 세션	서버가 로그인 상태를 구별하는 값 / 그 로그인 상태
계정 / 역할	Alice 같은 사용자 / 회원·관리자 같은 권한 조건
SCOPE / Origin	점검을 허용한 범위 / 프로토콜·호스트·포트의 조합
후보 / 증거	더 확인할 단서 / 실제 조건과 연결된 요청·응답
Repeater	요청을 편집하고 직접 다시 보내는 Burp 도구

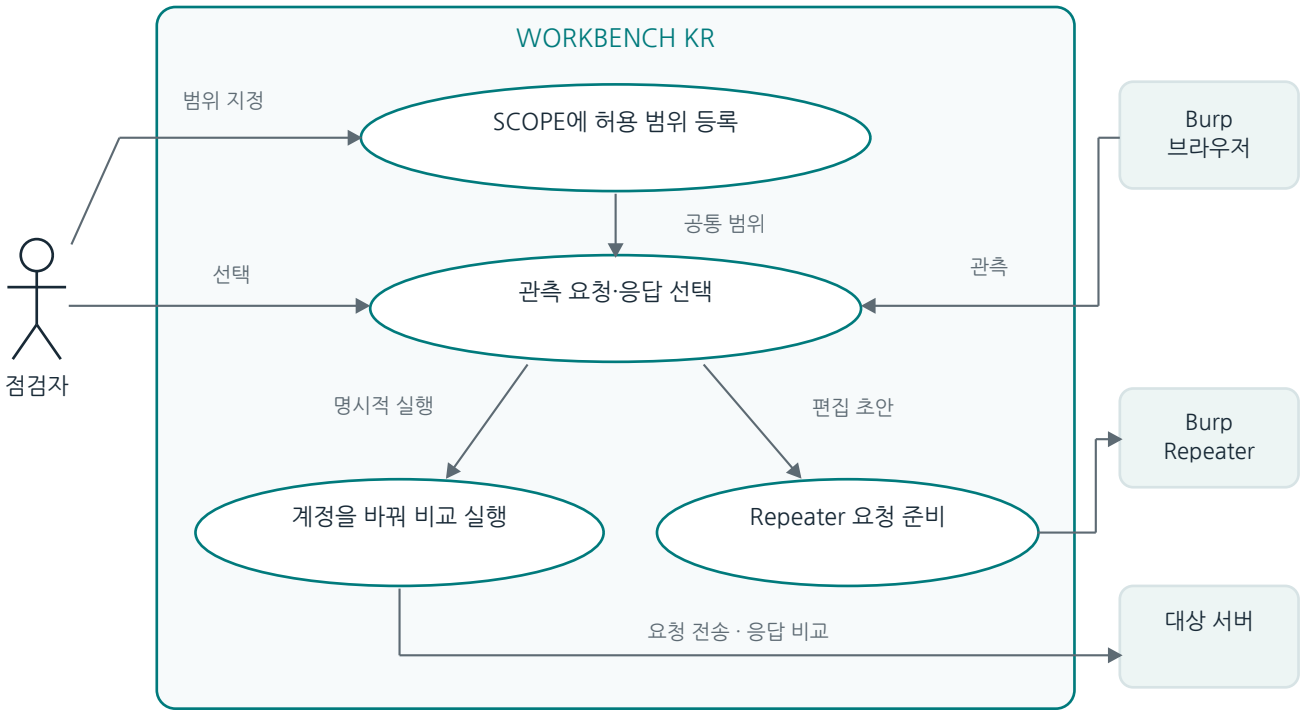


그림 1. Workbench 경계 안에서 요청을 연결하고 검사합니다. 외부 브라우저·서버·Repeater와의 자료 전달을 표시했습니다.

예: Bob과 Alice의 요청 주소가 같아도 로그인 쿠키가 다르면 서버 응답이 달라질 수 있습니다. HTTP 200은 응답을 받았다는 상태이며, 본문이 오류인지 실제 글인지도 읽어야 합니다.

03 · 설치하고 사이트 열기

준비물: Burp Suite, Workbench 0.25.0 JAR, Docker Desktop/Engine와 Compose v2. 안내서는 동봉한 로컬 실습을 기준으로 설명합니다.

1. downloads/README.md에서 0.25.0 JAR를 받습니다. GitHub 파일 화면의 다운로드 버튼으로 실제 파일을 저장합니다.

2. Burp에서 이전 Workbench를 해제하고 Extensions → Add → Java로 새 JAR 하나를 추가합니다.

3. 3.5.0 Docker 실행 ZIP을 별도 폴더에 풉니다. Windows는 start.ps1, Linux/macOS는 sh start.sh를 실행합니다.

4. 서비스 준비 후 Burp의 브라우저로 http://127.0.0.1:8088을 엽니다. 요청이 멈추면 Burp Intercept를 Off로 바꿉니다.

보여야 할 결과: Burp에 Workbench 탭이 나타나고 브라우저에 LOOP 커뮤니티가 열립니다. 처음에는 사이트 화면이 보이는지부터 확인하세요.

문제가 생기면	먼저 확인
JAR 대신 HTML이 저장됨	GitHub 파일 자체의 Download raw file 사용
확장 로드 오류	Extensions 오류 출력, JAR 버전, 현재 Burp 지원 여부
사이트 연결 실패	Docker 실행 상태, 포트 8088 충돌, 시작 로그
첫 실행이 오래 걸림	이미지·빌드 의존성 다운로드 진행 중인지 확인

직접 실행할 때는 docker compose up -d --build --wait --wait-timeout 900, 상태는 docker compose ps, 종료는 docker compose down입니다. down -v는 실습 데이터를 삭제하므로 단순 종료에 사용하지 않습니다.

다음: 5쪽에서 요청 한 건을 확인하고 점검 범위를 등록합니다.

04 · 첫 5분: 요청 한 건 확인하기

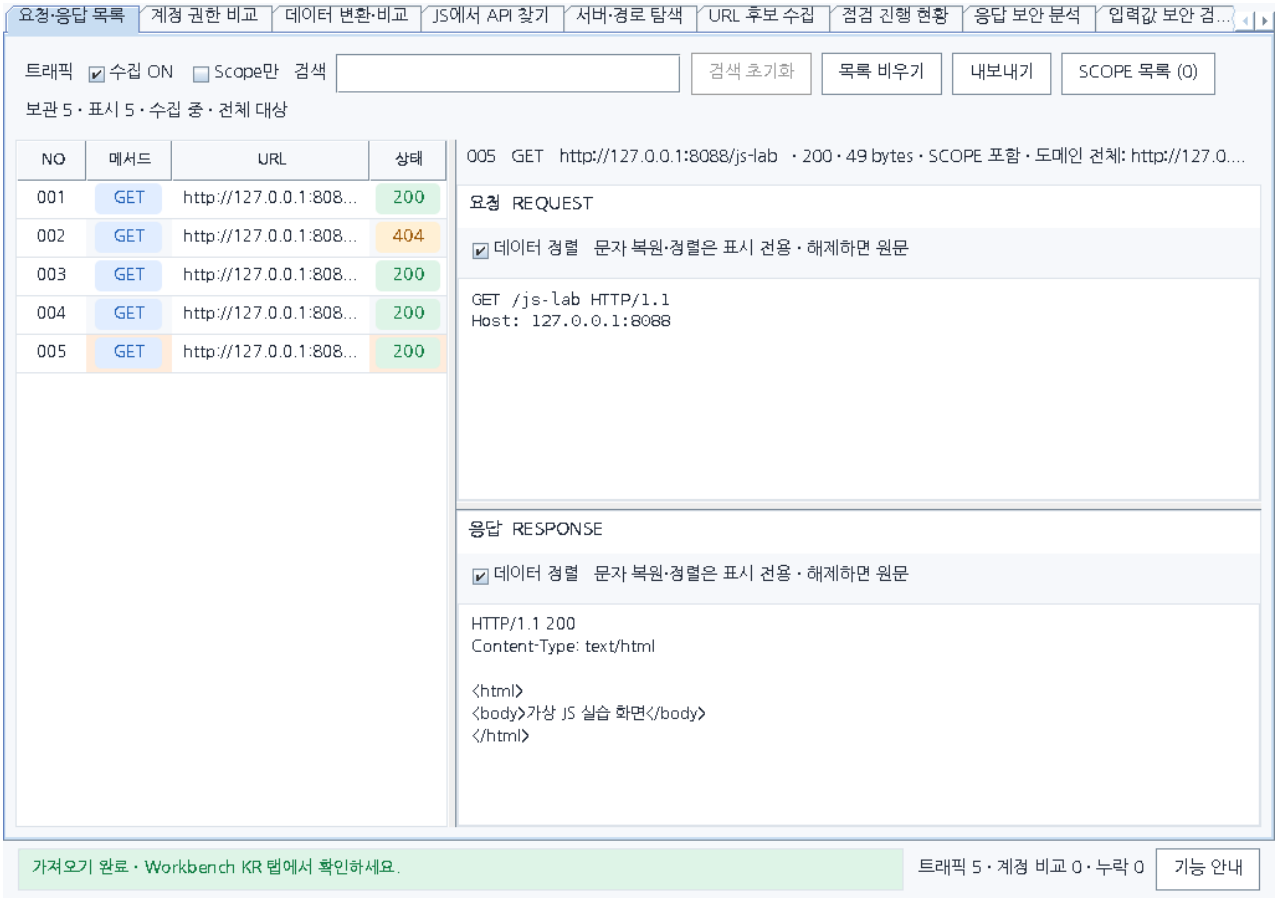


그림 2. 요청·응답 목록의 운영 UI 미리보기. 가상 요청을 넣었으며 실제 Burp 캡처가 아닙니다.

1. Burp 브라우저에서 실습 사이트의 게시글을 하나 엽니다. 요청·응답 목록에서 같은 주소 행을 선택합니다.

2. 행을 우클릭하여 SCOPE 지정 → 현재 사이트 전체 경로 추가를 누릅니다. SCOPE 목록에서 http://127.0.0.1:8088을 확인합니다.

3. 선택 행의 요청과 응답을 읽습니다. 메서드·주소·응답 상태·본문에서 어떤 글을 읽었는지 찾습니다.

4. 같은 요청을 직접 편집하려면 우클릭의 Repeater에 요청 준비를 사용합니다. Burp Repeater에서 내용을 확인하고 직접 Send를 누릅니다.

보여야 할 결과: 등록한 SCOPE와 선택 패킷이 보이고, Repeater에는 편집 가능한 요청 탭이 생깁니다. 전달만으로 새 요청을 보내지는 않습니다.

목록이 비면: Burp History에서 응답 있는 행을 선택해 Workbench KR → 요청·응답 목록에 가져오기를 사용합니다. localhost와 127.0.0.1은 다른 호스트입니다. Workbench SCOPE는 Burp Target Scope와 별개입니다.

05 · 서로 다른 로그인 두 개 만들기

중요: 같은 브라우저에서 Bob→Alice로 다시 로그인하면 이전 세션이 무효화될 수 있습니다. 비교용 세션은 아래처럼 Repeater에서 각각 새로 만듭니다. 모두 가짜 실습 계정이며 비밀번호는 LabOnly!2026입니다.

1. Burp 브라우저에서 한 번 로그인합니다. Burp History의 POST /api/auth/login 요청을 Repeater로 보내 Bob 로그인 탭을 만듭니다.

2. 요청의 Cookie 헤더 전체를 제거합니다. form 본문을 아래 값으로 바꾸고 Send합니다. 응답의 Set-Cookie: JSESSIONID=...에서 값만 복사합니다.

```
username=bob&password=LabOnly%212026
```

3. 원래 History의 로그인 POST를 다시 Repeater로 보냅니다. 새 탭에서 Cookie 헤더를 제거하고 Alice로 로그인합니다. 본문은 username=alice&password=LabOnly%212026이며 별도의 JSESSIONID를 보관합니다.

4. 각 계정의 확인용 요청은 아래처럼 GET으로 바꾸고 본문·Content-Length를 지웁니다. Bob값으로 Send한 뒤 별도 탭에 Alice값을 넣어 Send합니다. 응답에서 계정을 확인하고 두 요청을 Workbench 목록에 가져옵니다.

```
GET /api/auth/me HTTP/1.1
Host: 127.0.0.1:8088
Cookie: JSESSIONID=Bob의값
```

5. 계정 권한 비교의 인증 후보에서 Alice와 Bob 쿠키가 다른지 확인합니다. 점검 진행 현황 → 계정별 점검 → 계정 등록 후 각 /api/auth/me 요청을 골라 트래픽 연결하면 진행 기록도 계정별로 분리됩니다.

보여야 할 결과: /api/auth/me에서 Alice(ID 1)와 Bob(ID 2)이 각각 확인됩니다. 로그인 응답의 Set-Cookie만으로 인증 후보가 만들어지는 것은 아닙니다. 그 쿠키를 사용한 요청을 관측해야 합니다.

다음: 각 쿠키를 섞지 말고 다음 쪽에서 계정 슬롯에 지정합니다. 로그인 Cookie를 다시 보내 기존 세션을 재사용하지 않았는지도 확인하세요.

06 · 같은 글을 다른 계정으로 비교

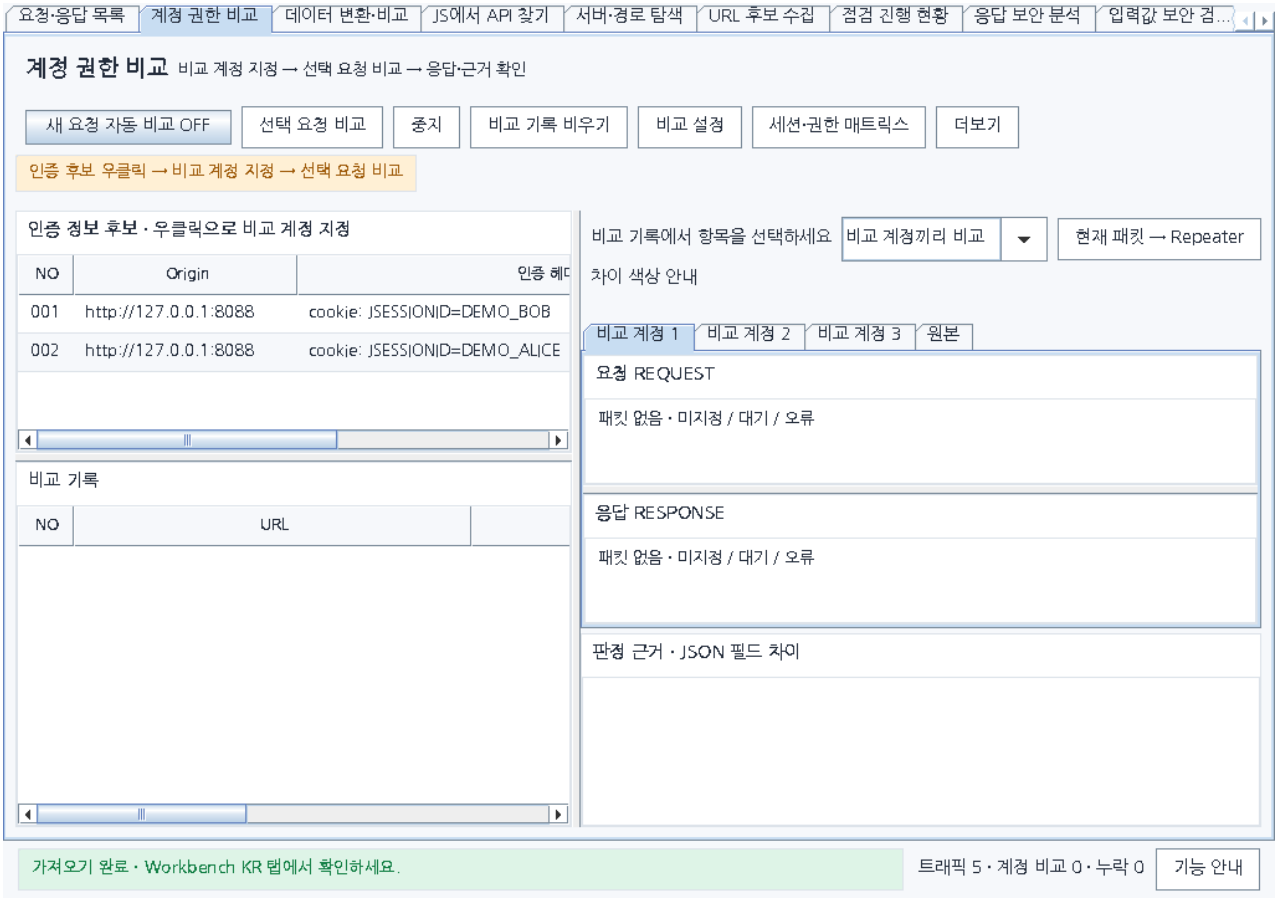


그림 3. 계정 권한 비교. 합성 데이터 미리보기이며 실제 비교 실행 결과가 아닙니다.

1. 앞에서 확인한 Alice JSESSIONID와 같은 Cookie 값의 후보 행 우클릭 → 비교 계정 1 → 선택 후보 지정, Bob 쿠키의 후보 행 → 비교 계정 2 → 선택 후보 지정을 선택합니다.

2. 비교 설정에서 이름 Alice/Bob, 소유값 1/2, 소유 JSON 경로 $.OWNER_ID를 지정합니다. 세션·권한 매트릭스의 기대 권한에는 Alice DENY, Bob ALLOW 조건을 둡니다.

3. Bob 쿠키의 GET /api/community/fixed/posts/102를 Repeater에서 보내 본문을 확인합니다. 응답 있는 요청을 Workbench 목록에 가져와 선택합니다.

4. 계정 권한 비교 → 선택 요청 비교를 누르고 비교 기록에서 비교 계정 1·2·원본 패킷을 읽습니다. vuln 경로도 같은 조건으로 대조합니다.

정책 입력 예: GET /api/community/fixed/posts/102 DENY ALLOW UNKNOWN. vuln도 동일한 기대 정책으로 비교합니다. 세 번째 슬롯은 처음에는 사용하지 않습니다.

보여야 할 결과: fixed는 Bob 200, Alice 404가 기대값입니다. vuln은 로그인 회원의 교차 읽기를 허용하는 학습 경로입니다. Bob 재조회와 본문의 OWNER_ID=2를 함께 확인합니다. 상태 숫자만으로 판정하지 않습니다.

다음: 증거 묶음에서 권한 비교 추가로 연결합니다. 처음에는 새 요청 자동 비교 OFF를 유지하세요.

07 · 입력값 하나를 바꿔 검사하기

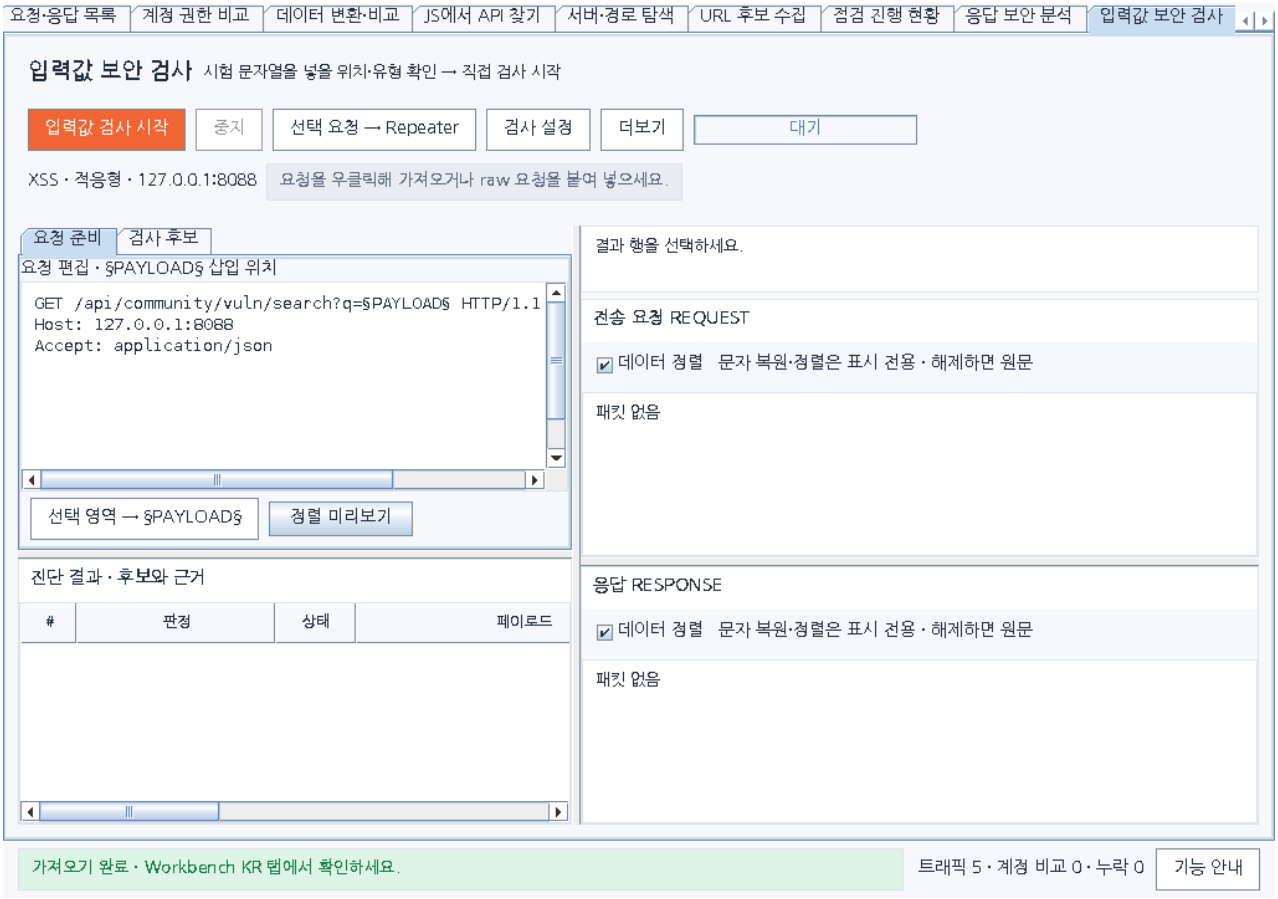


그림 4. 입력값 보안 검사의 실제 운영 패널. 이 미리보기는 검사를 실행하지 않았습니다.

예: 실습 사이트에서 검색한 정상 요청의 검색어 한 곳을 검사합니다.

1. /api/community/vuln/search?q=normal을 열어 얻은 정상 요청을 목록에서 선택하고 우클릭의 입력값 검사에 요청 연결로 가져옵니다.

2. 요청 준비에서 검색어 값만 선택하고 선택 영역 → §PAYLOAD§를 누릅니다. 주소·쿠키·다른 입력은 그대로 확인합니다.

3. 검사 설정에서 검사 유형, 대상, 요청 한도와 간격을 확인합니다. 예제는 SQL Injection, URL 인코딩, 기준값 normal, 총 한도 12으로 설정하고 유형 설명을 읽습니다.

4. 입력값 검사 시작을 누른 뒤 결과 행의 판정과 기준·검사·대조 응답을 읽습니다. 중간에 멈추려면 중지를 누릅니다.

보여야 할 결과: 요청 수·상태·시간·근거가 표시됩니다. 검사 후보를 바꿨다는 사실만으로 취약점이 확정되지는 않습니다. 로그인 만료·일시 지연도 비교 차이를 만들 수 있습니다.

다음: vuln 검사 종료 → 결과 행 선택 → 더보기 → 검사 결과 → 증거 묶음에 먼저 담습니다. 그다음 /api/community/fixed/search?q=normal을 같은 조건·한도 12로 검사하고 결과도 담아 ZIP을 저장합니다. 새 검사는 이전 검사 화면을 비우므로 먼저 증거로 옮기세요. 묶음 전체 한도는 32개입니다.

08 · JS에서 아직 보지 못한 API 찾기

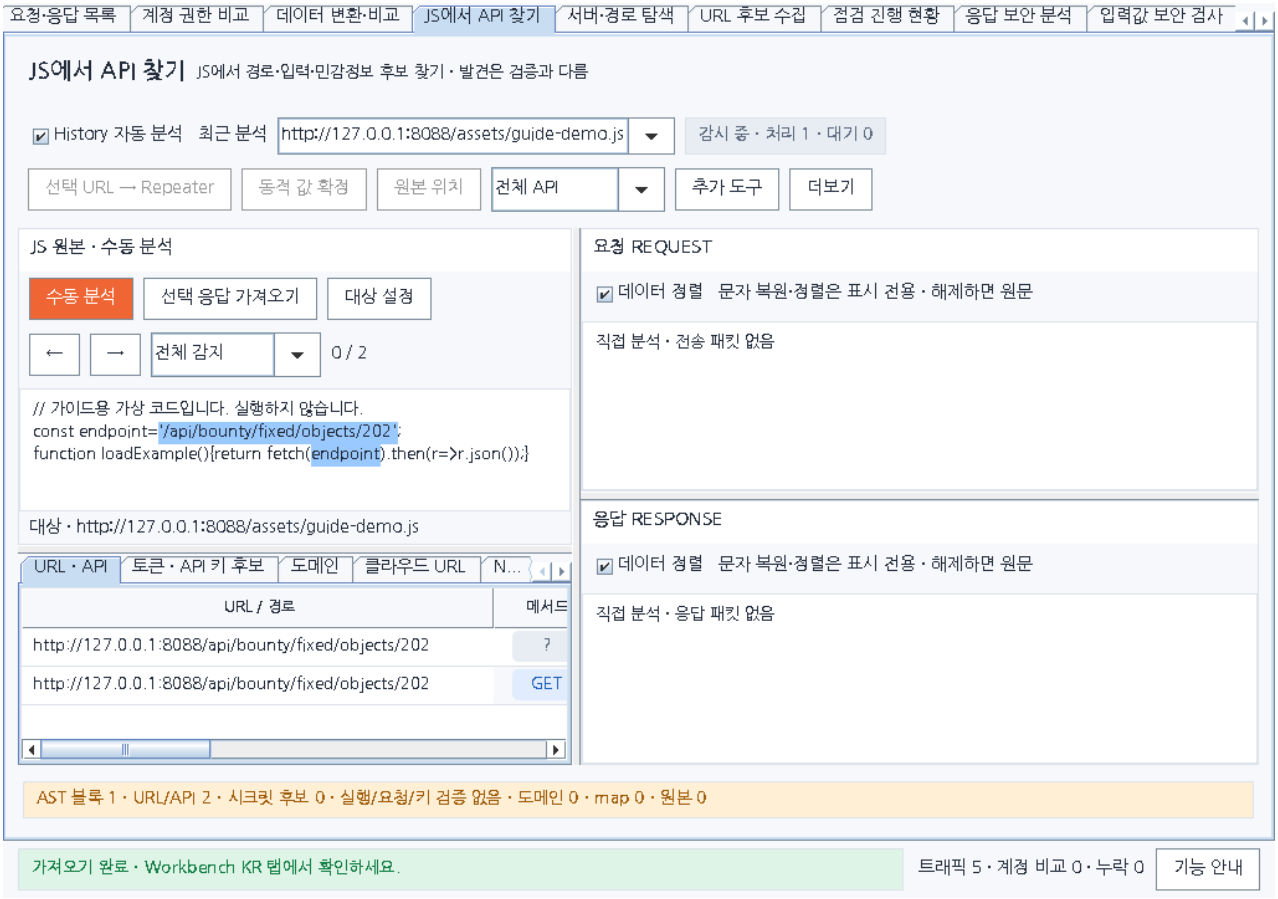


그림 5. JS에서 API 찾기. 소스와 후보 결과를 함께 보는 운영 UI 미리보기입니다.

1. 실습의 /js-lab을 열고 전체 수집 또는 샘플 선택 후 단일 수집을 누릅니다. 샘플 JS를 텍스트로 가져오며 실행하지 않습니다. Workbench SCOPE에 해당 사이트가 포함되어 있어야 합니다.

2. JS에서 API 찾기에서 최근 분석 출처를 고르고 URL · API 결과를 읽습니다. 수동 원문을 넣었다면 AST 분석을 실행합니다.

3. 결과 행을 선택하고 원본 위치로 이동해 호출 메서드·인증 조건·동적 변수를 확인합니다.

4. 확정된 경로는 Repeater 초안으로 준비합니다. 동적 부분이 있으면 동적 값 확정에서 실제 주소를 먼저 정합니다.

보여야 할 결과: URL·입력 이름·소스 줄 근거가 연결됩니다. 후보는 코드에서 찾은 문자열이며, 실제 API 호출 성공이나 취약점 판정은 아닙니다.

추가 도구: 소스맵 탐색은 범위 내 파일을 요청하고, NPM 존재 조회는 선택 레지스트리에 패키지 이름을 보냅니다. 수집 파일 저장·가림 결과 복사는 필요한 때 사용합니다.

다음: 요청 미관측 필터로 후보를 좁히고 점검 진행 현황에서 발견과 실제 관측을 구분합니다.

09 · 서버 단서와 경로 확인하기

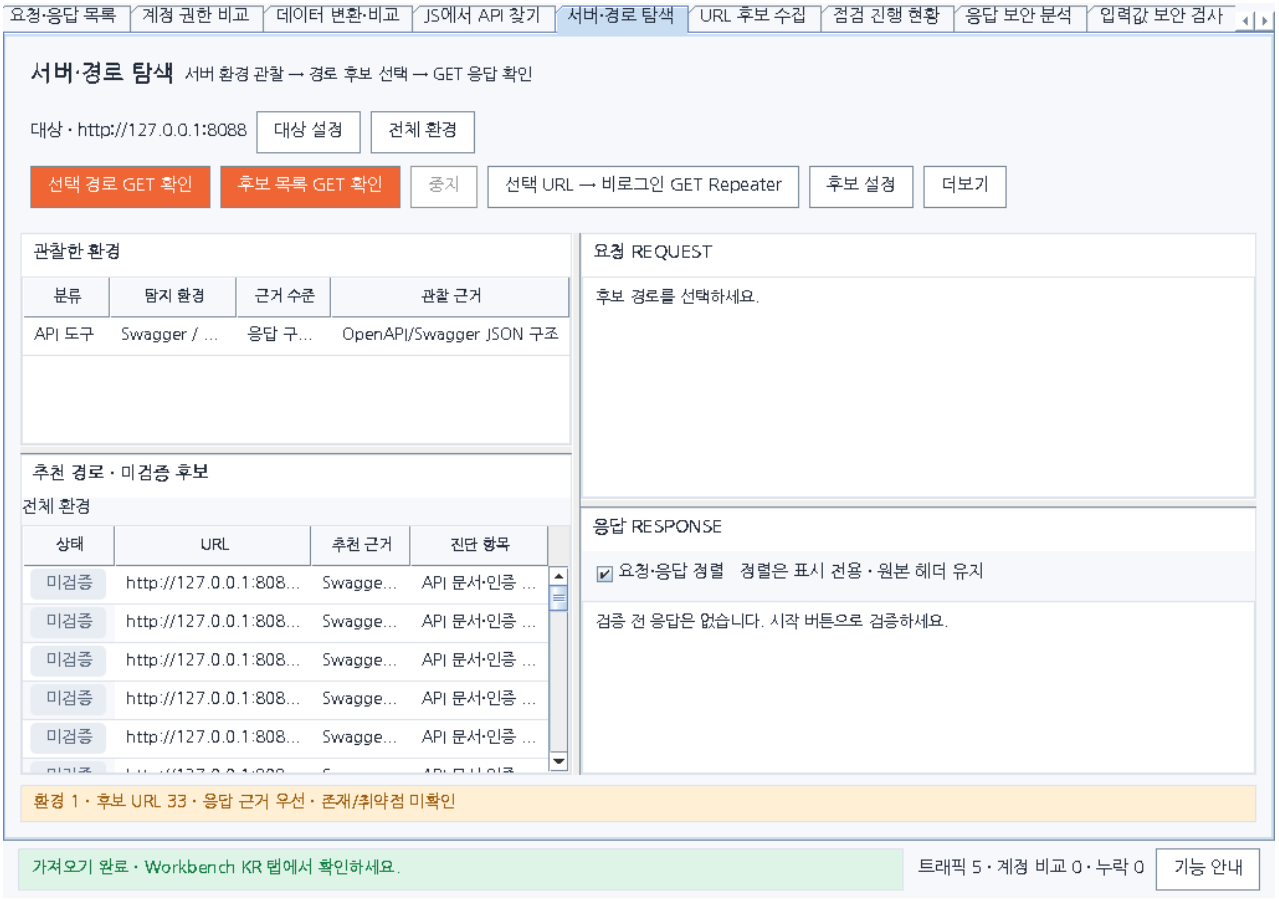


그림 6. 서버·경로 탐색. 관측 기술과 경로 후보는 검증 결과와 구분합니다.

이럴 때: 응답 헤더에 보이는 서버 기술을 바탕으로 점검할 관리·문서 경로를 고릅니다.

1. 사이트를 사용해 응답을 모은 뒤 서버·경로 탐색을 엽니다. 대상 설정에서 관측된 실습 Origin을 선택합니다.

2. 기술 목록의 관찰 근거를 읽고 경로 후보를 고릅니다. 이름만 보인 기술은 실제 사용 여부를 추가 확인합니다.

3. 후보 설정에서 계정을 확인합니다. 로그인 검증이 필요하면 미리 준비한 계정을 지정합니다.

4. 먼저 한 후보를 선택해 선택 경로 GET 확인을 누릅니다. 요청·응답과 없는 경로의 기준 응답을 비교합니다.

보여야 할 결과: 실제 요청 상태와 본문 비교가 표시됩니다. 없는 경로도 200을 돌려주는 서버가 있으므로 상태 숫자만으로 경로 존재를 판단하지 않습니다.

다음: 필요한 경로만 Repeater로 준비하거나 작은 목록으로 검사합니다. 후보 목록 GET 확인은 여러 요청을 보냅니다. 기준 안정성 확인 옵션도 추가 GET을 만들 수 있습니다.

10 · URL 후보를 모으고 줄이기



그림 7. URL 후보 수집. 출처와 검증 상태가 다른 열에 표시됩니다.

1. URL 후보 수집 탭 → 대상 설정에서 등록한 실습 SCOPE 선택 → 대상 열기를 누릅니다.

2. 수집 출처를 선택합니다. 로컬 실습은 Burp History / JS부터 시작하고 URL 수집을 누릅니다.

3. 검색·분류·패턴 대표 필터로 목록을 줄입니다. 과거 URL에는 현재 존재하지 않는 경로도 섞일 수 있습니다.

4. 확인할 행 하나를 고르고 선택 URL GET 확인으로 GET 검증합니다. 실제 원본이 생기면 선택 검증 원본 → 증거로 옮길 수 있습니다.

보여야 할 결과: 발견 출처와 실제 GET 상태를 따로 읽을 수 있습니다. 수집된 URL은 실행한 요청과 다르며, 발견 메서드가 POST면 GET 결과만으로 기능을 판정하지 않습니다.

수집 주의: 요청 목록의 URL 후보 수집 시작은 수집까지 실행합니다. 로컬 입문에서는 위 순서로 출처부터 확인하세요. 표시 URL GET 확인은 보이는 여러 URL에 요청합니다.

고급 사용: Wayback·waybackurls·gau는 외부 자료/설치 도구를 사용합니다. URL DB 저장·이전 DB 가져오기는 새 경로를 비교할 때, 패턴 요약은 대표 요청을 고를 때 씁니다.

다음: 대표 요청의 정책과 입력을 검토하고 진행 현황에 기록합니다. 삭제·수정처럼 상태를 바꿀 수 있는 경로는 GET 이름만 보고 일괄 실행하지 않습니다.

11 · 아직 점검하지 않은 부분 찾기



그림 8. 점검 진행 현황. 발견·관측·완료를 구분하는 운영 화면입니다.

1. 점검 진행 현황 → URL 점검 현황에서 미완료 URL 또는 입력 필터를 선택합니다.

2. URL 행을 고르고 입력 목록을 확인합니다. URL, 본문, 헤더, 쿠키, 숫자 경로는 서로 다른 입력 위치입니다.

3. JS에서 찾은 후보라면 발견 근거 또는 소스 근거로 원문과 호출 조건을 확인합니다.

4. 실제 요청을 검토한 뒤 입력 완료, URL 완료를 직접 누릅니다. 아직 요청하지 않은 후보는 미완료로 둡니다.

보여야 할 결과: 실제 확인한 대상만 완료로 표시됩니다. 요청을 관측했거나 검사를 실행한 횟수가 수동 완료 횟수와 같지는 않습니다.

보관: 기록 에서 기록 저장·불러오기·Excel 내보내기를 사용합니다. 저장 파일은 진행 메타데이터이며 로그인 세션을 복원하지 않습니다.

다음: 계정에 따라 정책이 다르면 계정별 점검으로 옮겨 해당 계정만 완료합니다. 숫자 경로를 찾았다는 사실만으로 타인 소유 데이터 접근이 증명되지는 않습니다.

12 · 작은 업무로 묶어 누락 막기

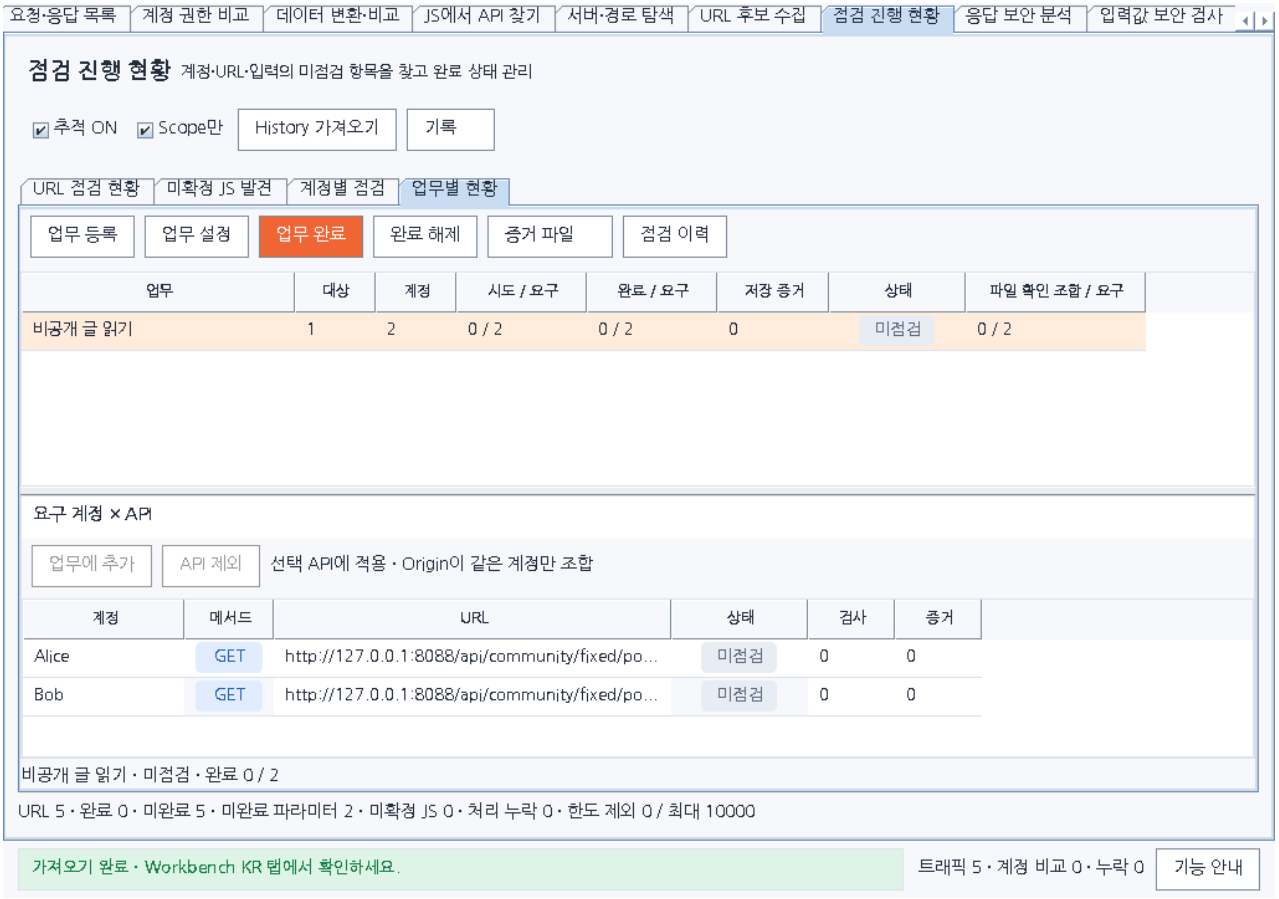


그림 9. 업무별 현황의 계정×API 조합. 가상 계정으로 구성한 미완료 업무입니다.

예: '비공개 글 읽기'를 Bob·Alice 두 계정으로 점검할 업무로 만듭니다.

1. 점검 진행 현황 → 업무별 현황 → 업무 등록에서 제목을 쓰고 필요한 계정을 선택합니다.

2. 계정별 점검 또는 URL 목록에서 대상 API를 고르고 업무에 추가를 누릅니다. 같은 Origin의 계정만 조합됩니다.

3. 업무 설정에서 '모든 입력 수동 완료 필수', '모든 계정×API의 파일 확인 증거 필수'를 필요한 수준으로 정합니다.

4. 중요 입력에는 URL:id, BODY:ownerId처럼 바뀌면 다시 봐야 할 항목만 넣습니다. 정상 요청을 한 번 관측해 기준을 잡습니다.

보여야 할 결과: 업무 아래에 Bob×API, Alice×API 조합과 각각의 상태가 보입니다. 계정 하나를 완료해도 다른 계정은 완료되지 않습니다.

다음: 검사와 증거 저장을 마친 뒤 22쪽에서 업무를 마감합니다. 페이지 번호처럼 자주 바뀌는 값까지 중요 입력으로 지정하면 불필요한 재검토가 늘어납니다.

13 · 이미 받은 응답에서 단서 찾기



그림 10. 응답 보안 분석의 응답 관찰. 실제 추가 요청 없이 받은 응답을 읽습니다.

이럴 때: 응답 헤더, 쿠키 속성, 오류 문구처럼 이미 받은 내용에 확인할 단서가 있는지 봅니다.

1. 실습 사이트를 이용한 뒤 응답 보안 분석 → 응답 관찰을 엽니다.

2. 행을 선택해 규칙·근거 위치·관찰 문구를 읽고, 아래 영향 URL 목록을 확인합니다.

3. 실제 응답에서 해당 헤더나 문구가 보이는지, 서비스 동작상 어떤 의미인지 확인합니다.

4. 원본이 연결되어 있으면 선택 관찰 → 증거로 대표 요청·응답을 전달합니다.

보여야 할 결과: 관찰 근거와 실제 원본이 연결됩니다. 같은 그룹의 대표 요청 하나가 모든 영향 URL의 상태까지 입증하지는 않습니다.

예: 응답에 오류 클래스 이름이 있어도 학습용 고정 문자열일 수 있습니다. 실제 예외가 발생한 조건과 노출 범위를 확인한 뒤 영향 설명을 씁니다.

다음: 문서로 발견한 API는 다음 쪽에서 별도로 확인합니다. 원본이 만료되었거나 너무 크면 해당 동작을 다시 관측합니다.

14 · API 문서를 요청 초안으로



그림 11. API 문서의 메서드·경로·입력 목록. 문서 후보와 실제 관측을 구분합니다.

이럴 때: OpenAPI/Swagger 문서에 있는 경로를 실제 요청으로 확인하려고 합니다.

1. Burp 브라우저로 실습의 /v3/api-docs를 열어 문서 응답을 관측합니다.

2. 응답 보안 분석 → API 문서에서 메서드와 경로를 고릅니다. 관찰 결과 열에서 실제 요청이 있었는지 확인합니다.

3. Repeater 초안을 누르고 {id} 같은 경로 변수를 실습의 실제 값으로 채웁니다.

4. Repeater 초안 준비 후 Burp Repeater에서 인증·본문·입력값을 확인하고 직접 전송합니다.

보여야 할 결과: 문서 메서드와 입력 이름을 반영한 요청 초안이 생깁니다. 인증값이나 민감한 쿼리 값을 자동 복사하지 않으므로 필요한 실제 값을 확인해야 합니다.

예: /posts/{id}를 /posts/102로 바꿔도 그 글의 소유자나 접근 정책은 별도로 확인합니다. 문서가 오래되었으면 실제 메서드가 다를 수 있습니다.

다음: 응답을 관측하고 점검 진행 현황에서 검토합니다. 파일 업로드처럼 multipart 본문이 필요한 요청은 브라우저가 만든 실제 트래픽을 사용하세요.

15 · 저장한 값이 어디에 나타나나요?

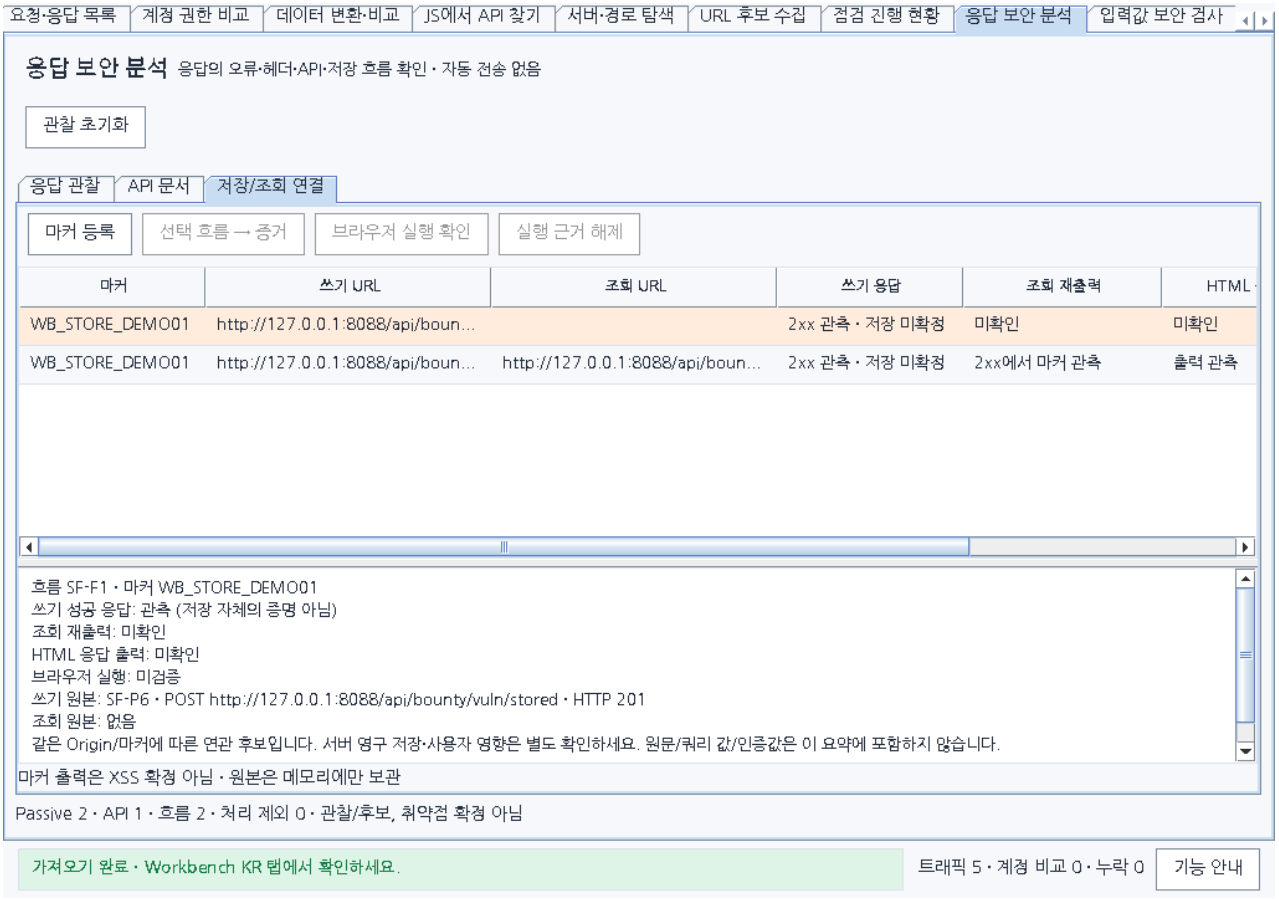


그림 12. 저장·조회 단계의 가상 예시. 브라우저 실행 확인은 하지 않은 상태입니다.

1. 응답 보안 분석 → 저장/조회 연결 → 마커 등록에서 WB_STORE_DEMO01처럼 다른 입력과 구별되는 표시를 등록합니다.

2. 실습 /bounty-lab의 저장 입력에 그 문자열을 넣고 저장합니다. 이어서 목록·상세·반환된 조회 URL을 엽니다.

3. 연결 목록에서 쓰기 응답, 조회 재출력, HTML 출력, 브라우저 실행을 각각 읽습니다.

4. 선택 흐름 → 증거로 쓰기와 조회 원본을 함께 옮깁니다.

보여야 할 결과: 같은 마커가 쓰기와 후속 조회에 연결됩니다. 문자열 재출력, HTML 출력, 스크립트 실행은 서로 다른 사실입니다.

브라우저 확인: 실제 브라우저에서 같은 흐름의 실행을 직접 확인한 경우에만 브라우저 실행 확인으로 가린 PNG/JPEG를 연결합니다. 이미지가 있다는 이유만으로 실행을 확인했다고 표시하지 않습니다.

다음: 영향과 재현 순서를 증거 묶음에 적습니다. 새 쓰기나 새 조회가 생기면 기존 실행 근거가 여전히 같은 조건인지 다시 확인합니다.

16 · 변환·JWT·응답 차이 보기

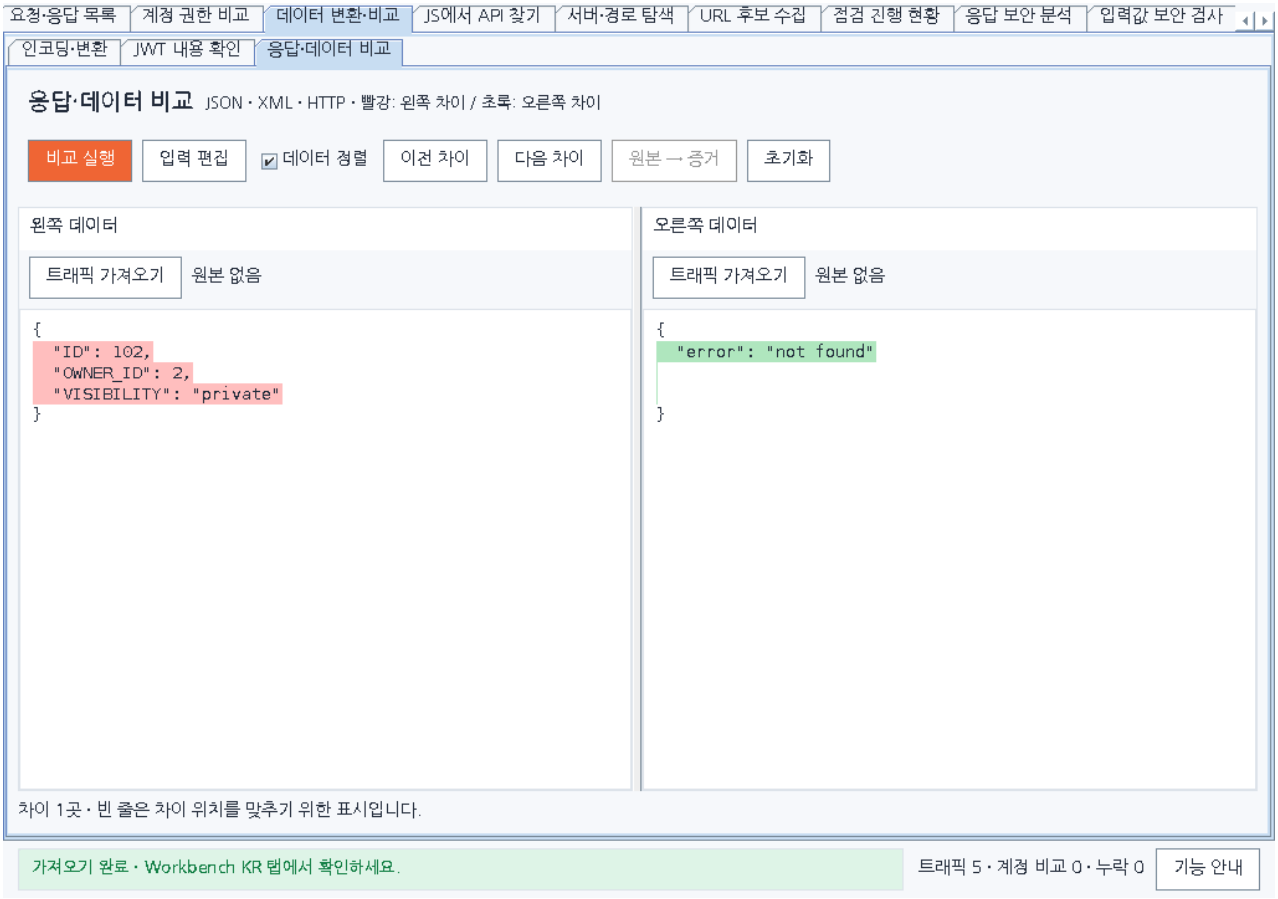


그림 13. 데이터 비교의 가상 입력. 직접 붙여넣은 내용에는 원본 패킷 연결이 없습니다.

목적	순서	보여야 할 결과
URL·Base64·Hex·해시·압축 변환	인코딩·변환 탭 → 입력 → 작업 선택 → 변환	출력 확인. 필요하면 출력 → 입력으로 다음 변환
JWT 내용 읽기	JWT 내용 확인 탭 → 선택 요청에서 가져오기 또는 직접 입력 → JWT 디코딩	헤더·페이로드·시간 정보. 서명 검증 결과는 아님
두 응답 비교	기준 요청 선택 → 왼쪽 트래픽 가져오기 → 대조 요청 선택 → 오른쪽 가져오기 → 비교 실행	구조·본문 차이와 각각의 출처

예: Bob의 정상 글 JSON과 Alice의 차단 JSON을 비교합니다. 실제 패킷을 가져왔으면 원본 → 증거로 두 원본을 함께 전달할 수 있습니다.

주의할 차이: 입력을 편집하면 원본 연결이 해제됩니다. 붙여넣은 텍스트는 원본 증거가 아닙니다. 표시 비교는 64 KiB/1,500줄, 연결 원본은 요청·응답 합계 128 KiB 이하입니다.

다음: 차이 자체가 아니라 그 차이로 확인한 접근 정책·노출 내용을 기록합니다. JWT의 role 문자열만으로 서버 권한 상승을 결론 내리지 않습니다.

17 · 여러 결과를 한 증거로 연결

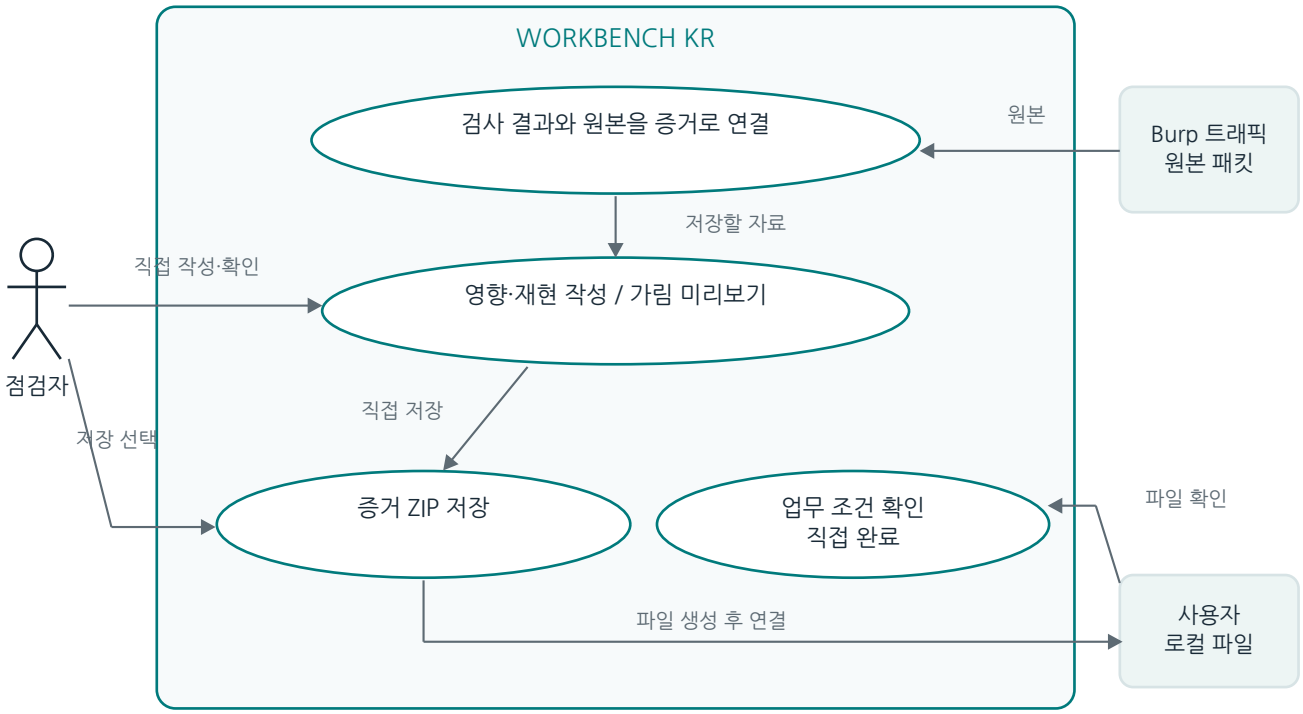


그림 14. 검사 결과·원본을 모아 저장하고, 저장한 파일을 업무 기록과 연결하는 사용 사례입니다.

두 종류의 기록: 검사 결과는 방금 실행한 요청의 관찰입니다. 업무 완료는 필요한 계정·입력·증거를 사람이 검토했다는 표시입니다. 한쪽이 생겼다고 다른 쪽이 자동 완료되지는 않습니다.

1. 계정 비교·입력값 검사·응답 분석에서 필요한 원본을 증거 묶음으로 보냅니다.
2. 여러 요청을 같은 순서로 재현해야 하면 다음 쪽의 업무 흐름을 준비합니다.
3. 실제 영향과 재현 조건을 적고, 저장될 내용을 가림 미리보기로 확인합니다.
4. ZIP 저장 후 파일을 확인하고 해당 업무의 완료 조건을 점검합니다.

예: Bob의 정상 읽기, Alice의 취약 경로 읽기, Alice의 수정 경로 차단을 한 묶음으로 보관합니다. 세 요청의 역할을 재현 문장에 적으면 다른 사람이 결과를 읽기 쉽습니다.

다음: 순차 재현은 19쪽, 증거 저장은 20쪽, 업무 마감은 22쪽입니다.

18 · 여러 요청을 순서대로 재현

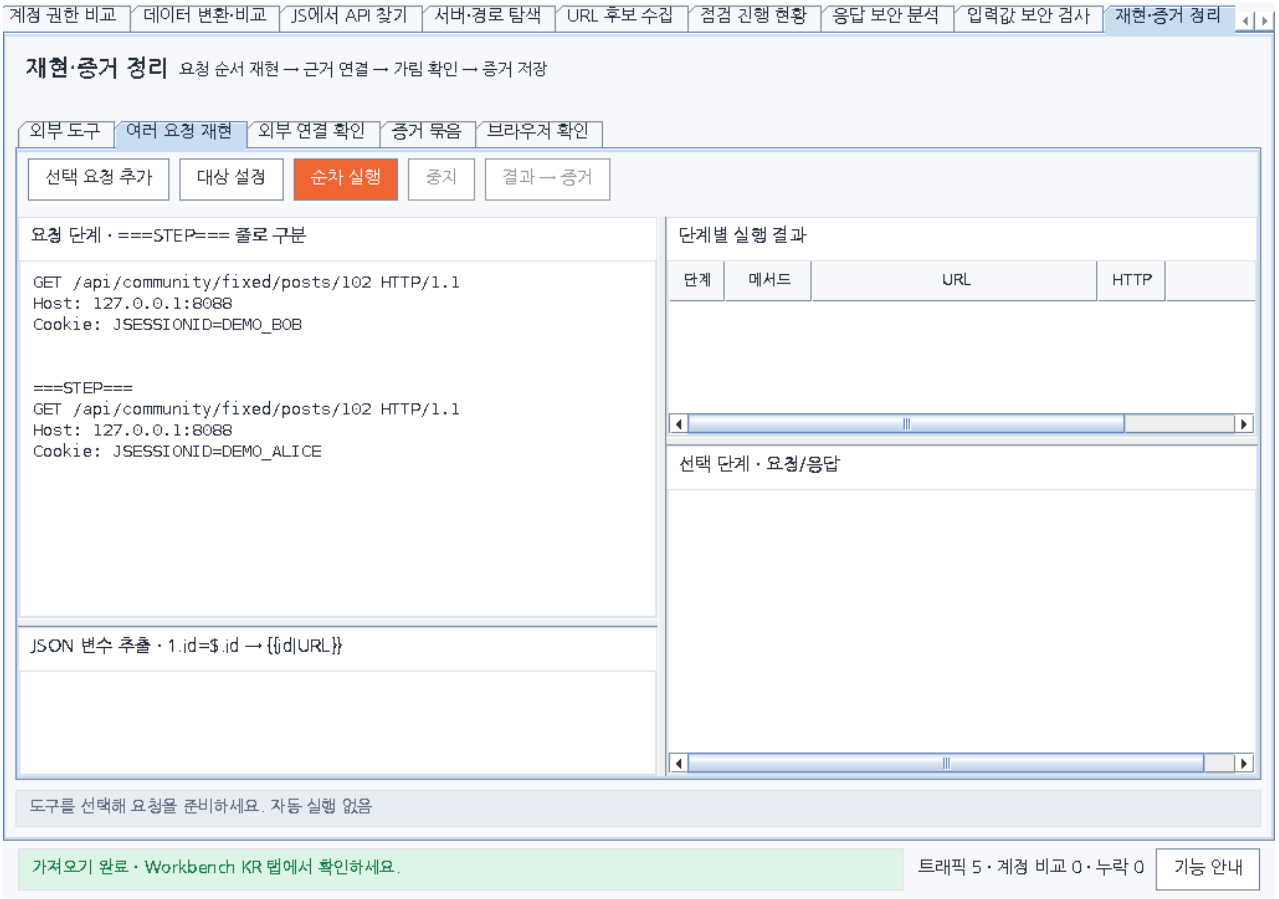


그림 15. 업무 흐름의 운영 화면. 요청 초안만 준비했으며 실행하지 않았습니다.

예: 게시물을 만든 응답에서 ID를 얻고, 그 ID로 다시 조회하는 두 단계입니다.

1. 재현·증거 정리 → 여러 요청 재현에서 선택 요청 추가로 첫 요청을 넣습니다. 다음 요청도 선택해 추가합니다. 구분 줄은 ===STEP===입니다.

2. 대상 설정에서 Origin·쓰기 단계 허용·요청 한도·기대 HTTP/JSON 조건을 확인합니다.

3. 응답 변수는 1.id=$.id, 다음 URL에는 {{id|URL}}처럼 지정합니다. JSON 문자열은 {{id|JSON}}, 그대로 넣을 때만 RAW를 사용합니다.

4. 순차 실행 후 단계별 요청·응답·중단 이유를 읽습니다. 검사 종료 뒤 결과 → 증거로 보냅니다.

보여야 할 결과: 성공한 이전 응답의 값이 다음 단계에 사용됩니다. 기대 조건이나 변수 추출이 실패하면 후속 단계를 중단합니다.

처음에는: 변수 없이 두 개의 정상 GET 요청으로 순서를 익히세요. 최대 8단계이며 같은 Origin/SCOPE에서 리디렉션 없이 실행합니다. POST 등 쓰기는 실제 실습 데이터를 바꿀 수 있습니다.

19 · 읽을 수 있는 증거 ZIP 저장

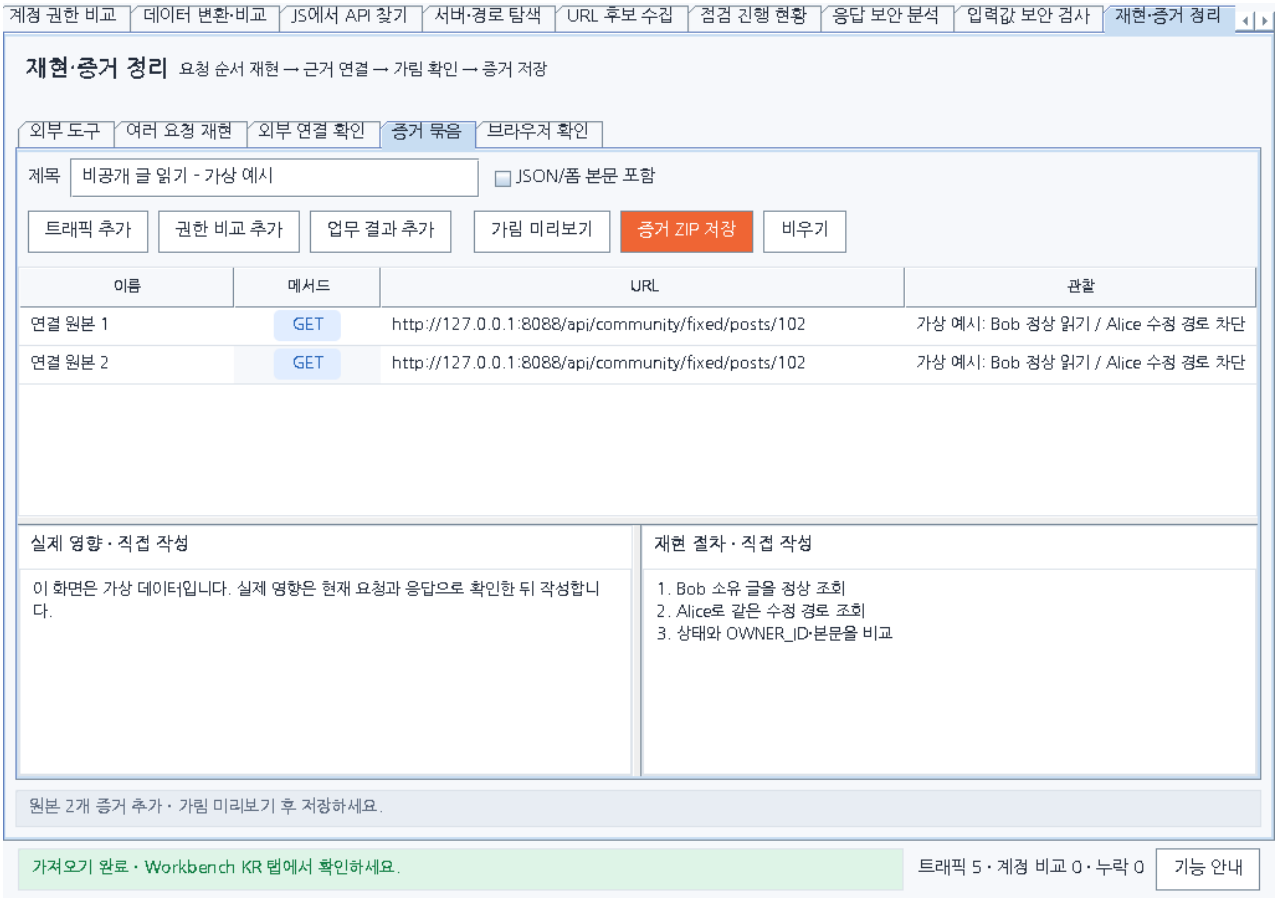


그림 16. 증거 묶음의 가상 예시. 영향·재현 문장은 직접 작성합니다.

1. 재현·증거 정리 → 증거 묶음에서 트래픽·권한 비교·업무 결과를 추가하거나 다른 도구에서 보낸 원본을 확인합니다.

2. 제목에 기능과 영향 하나를 적습니다. 영향에는 '누가 어떤 정보를 볼 수 있었는지', 재현에는 계정·대상·순서·기대/실제를 적습니다.

3. 가림 미리보기에서 인증·쿼리·민감 키와 직접 쓴 설명을 확인합니다. JSON/폼 본문 포함은 필요한 경우만 선택합니다.

4. 증거 ZIP 저장을 누르고 파일이 실제 생성됐는지 확인합니다.

보여야 할 결과: 요청·응답과 설명을 담은 ZIP이 생깁니다. 준비된 목록에 보이는 상태와 디스크에 저장된 상태는 다릅니다.

보관 한도: 한 묶음은 요청 32개와 이미지 1장까지입니다. 다른 이미지가 필요하면 현재 묶음을 저장하고 비운 뒤 새로 만듭니다. 가린 ZIP만으로 실제 인증을 재현할 수는 없습니다.

다음: 22쪽에서 파일 상태와 업무 완료 조건을 확인합니다.

20 · OAST·외부 도구·브라우저 연결

이럴 때: 대상 응답만으로 보이지 않는 외부 상호작용을 확인하거나, Burp의 다른 전문 도구와 결과를 연결하려고 합니다.

하려는 일	클릭 순서	결과와 다음 행동
Collaborator 주소 준비	요청 선택 → 재현·증거 정리 → 외부 연결 확인 → 콜백 ID 발급	요청과 연결된 주소 발급. 대상 요청은 별도로 준비·실행
돌아온 콜백 보기	같은 화면 → 콜백 조회	종류·시각·ID를 원래 요청과 대조
외부 OAST 기록 연결	외부 결과 형식에 맞게 붙여넣기 → 외부 콜백 결과 가져오기	ID·METHOD·URL이 같은 실험인지 검토
외부 BApp과 작업 연결	외부 도구 → 도구 선택 → 선택 요청 → Repeater	안내에 따라 별도 도구 실행, 입력 이름 결과만 활동 기록에 연결
브라우저 근거 첨부	브라우저 확인 → 동일 기능 수동 재현 → 브라우저 증거 이미지 첨부	가린 PNG/JPEG를 증거 묶음에 연결

용어: OAST는 대상이 외부 주소에 접속했는지 관찰하는 방식입니다. 콜백은 그 상호작용 기록입니다. DNS 기록 하나만으로 서버 코드 실행 같은 영향을 확정하지 않습니다.

준비: Collaborator 연동은 사용 가능한 Burp 서비스가 필요합니다. 별도 도구는 직접 설치한 원래 확장으로 사용합니다. 이 가이드의 합성 화면은 실제 Pro 서비스·외부 BApp·브라우저 실행 검증이 아닙니다.

다음: 직접 확인한 상호작용, 재현 조건, 확인하지 못한 영향을 나눠 적고 증거를 저장합니다.

21 · 완료 표시하고 다음 작업 남기기

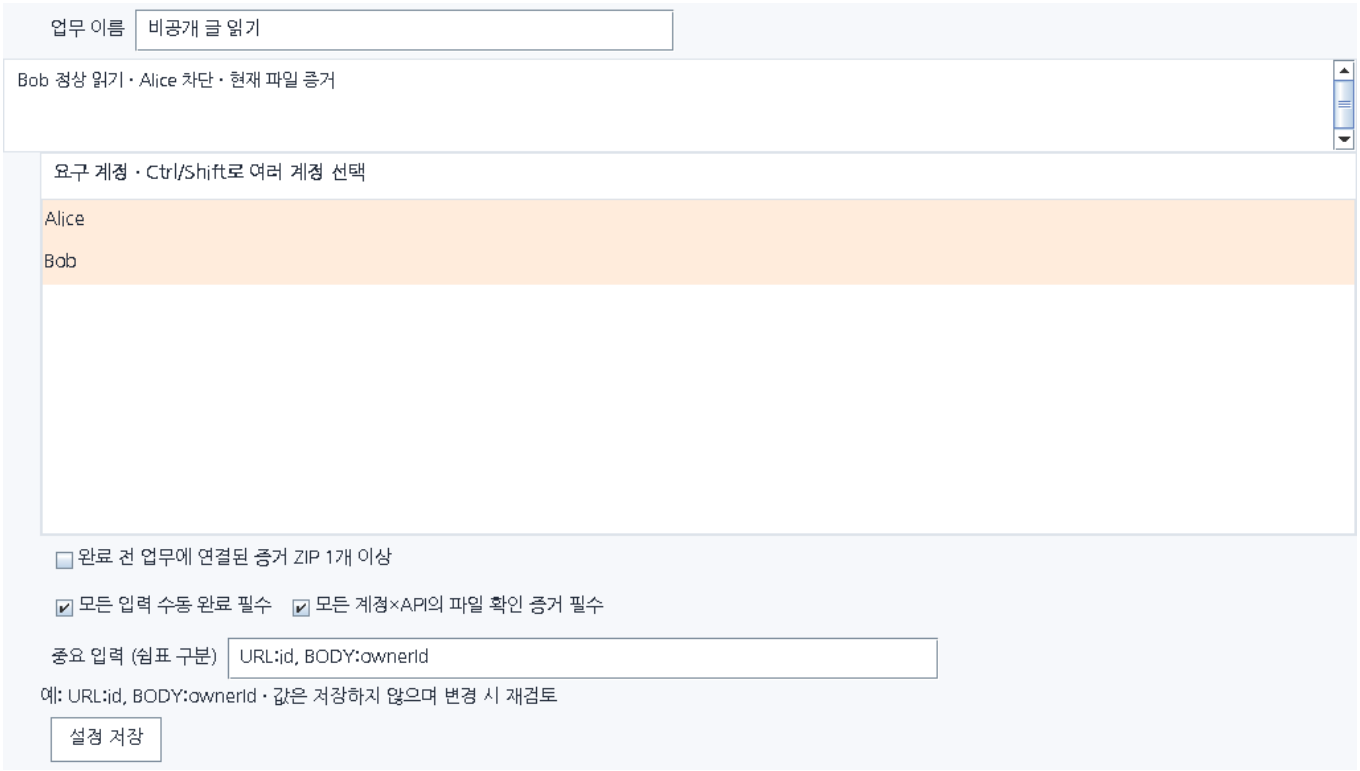


그림 17. 업무 완료 조건. 필요한 입력과 파일 증거를 미리 정합니다.

1. 계정별 점검에서 해당 계정의 API·입력을 검토하고 입력 완료 / 해제, URL 완료로 표시합니다.

2. 업무별 현황에서 요구 계정×API의 완료와 파일 확인 증거를 확인합니다. 부족한 조합을 먼저 해결합니다.

3. 증거 파일 → 모든 증거 재검증으로 현재 파일 상태를 확인합니다. 파일을 이동했으면 증거 ZIP 재연결을 사용합니다.

4. 조건을 만족하면 업무 완료를 직접 누릅니다. 계정 항목을 재완료해도 업무는 자동 마감되지 않습니다.

보여야 할 결과: 요구 조합과 선택한 입력·증거 조건이 충족된 업무만 완료됩니다. 역할·조직·중요 입력·대상이 달라지면 재검토가 필요할 수 있습니다.

다음에 이어 하기: 기록 JSON을 불러와도 세션·로컬 파일 경로·중요 입력 기준 해시는 복원하지 않습니다. 로그인 연결과 파일 근거를 다시 확인하세요. 파일 상태는 마지막 수동 확인 결과입니다.

22 · 결과를 짧게 쓰는 예

예시일 뿐, 실제 검증 결과가 아닙니다. 현재 실습의 요청·응답과 계정 조건을 확인한 뒤 자기 결과로 바꿔 씁니다.

항목	작성 예
제목	일반 회원이 다른 회원의 비공개 글을 읽을 수 있음
조건	Bob 소유 글 102, Alice 일반 세션, 로컬 vuln 경로
순서	Bob 정상 읽기 → Alice로 같은 글 읽기 → fixed 경로 대조
기대 / 실제	소유자만 읽어야 함 / Alice 응답 본문에 Bob의 글 내용 표시
영향	타 회원의 비공개 정보 노출
증거	각 계정의 요청·응답, 현재 시각, ZIP 파일
확인하지 않은 것	실제 서비스 영향·운영 데이터·별도 브라우저 실행

막힐 때 가장 먼저 볼 곳

증상	확인 순서
목록이 비어 있음	응답 있는 요청인가 → 수집 ON인가 → Scope만/검색 필터인가
버튼이 비활성화됨	올바른 행 선택 → 원본 보관 상태 → 실행 중인지 확인
두 계정 결과가 같음	실제 로그인 주체 → 슬롯 쿠키 → 대상·본문·기대 정책
200인데 결과가 이상함	로그인 페이지·오류 JSON·없는 경로 공통 응답인지 확인
증거를 저장할 수 없음	원본 요청 추가 → 검사 종료 → 미리보기 → 저장 위치

화면 하단 기능 안내에서 현재 작업의 시작 조건을 다시 볼 수 있습니다. 결과가 달라지면 데이터·세션·입력값을 기록하고 같은 조건으로 재확인하세요.

23 · 이전 이름으로 새 기능 찾기

기존 기능은 유지하면서, 무엇을 하는 화면인지 알아보기 쉽게 이름을 바꿨습니다. 이전 이름이 익숙하면 아래 표에서 새 위치를 찾으세요.

이전 이름	0.25.0 이름	하는 일
트래픽	요청·응답 목록	관측 요청과 응답 보기
권한 비교	계정 권한 비교	같은 요청을 계정별로 비교
데이터 도구	데이터 변환·비교	인코딩·JWT·응답 차이 확인
JS 추출	JS에서 API 찾기	코드의 경로·입력 후보 찾기
환경 탐색	서버·경로 탐색	서버 단서와 경로 확인
URL 수집	URL 후보 수집	여러 출처의 URL 모으기
활동 추적	점검 진행 현황	URL·입력·계정·업무 상태 관리
응답 분석	응답 보안 분석	응답 단서·API 문서·저장 흐름
페이로드 검사	입력값 보안 검사	한 입력을 바꿔 응답 비교
점검 지원	재현·증거 정리	순서대로 재현하고 증거 저장

하위 화면도 바뀌었습니다: 변환 → 인코딩·변환, JWT → JWT 내용 확인, 비교 → 응답·데이터 비교, Passive 관찰 → 응답 관찰, 업무 흐름 → 여러 요청 재현, OAST 연결 → 외부 연결 확인입니다.

어디서든 찾기: 하단 기능 안내에서 기능 이름을 선택하고 이 기능 열기로 바로 이동합니다. 안내 화면을 열거나 이동하는 것만으로 검사를 시작하지 않습니다.

이 안내서의 기준: 확장 0.25.0 · 실습 사이트 3.5.0. 그림은 현재 운영 Swing 코드와 가상 데이터로 만든 미리보기입니다. 실제 Burp·Docker·브라우저·Pro 서비스의 실행 확인은 docs/VERIFICATION_0.25.0.json과 해당 PR의 기록에서 구분해 읽습니다. 개발·재생성 명령은 DEVELOPMENT_SETUP_KR.md에 있습니다.